

ĐẠI HỌC BÁCH KHOA HÀ NỘI
TRƯỜNG CÔNG NGHỆ THÔNG TIN VÀ TRUYỀN THÔNG



BÁO CÁO BÀI TẬP LỚN
MÔN HỌC: CÔNG NGHỆ WEB VÀ DỊCH VỤ TRỰC TUYẾN

GVHD: TS. Đỗ Bá Lâm

Nhóm sinh viên thực hiện

Họ và tên	MSSV
Nguyễn Nhật Huy	20225335
Nguyễn Quang Huy	20225336
Nguyễn Văn Quang Huy	20225337
Trần Quang Huy	20225201

Hà Nội, 12/2025

LỜI NÓI ĐẦU

Trong bối cảnh đô thị hóa nhanh chóng hiện nay, ùn tắc giao thông đang trở thành một vấn đề cấp thiết, đòi hỏi những giải pháp công nghệ thông minh và hiệu quả hơn. Sự phát triển mạnh mẽ của Trí tuệ nhân tạo và các công nghệ xử lý dữ liệu thời gian thực đang mở ra hướng đi mới cho việc giải quyết bài toán này.

Trong khuôn khổ môn học Công nghệ Web và dịch vụ trực tuyến, nhóm sinh viên chúng em đã lựa chọn và thực hiện đề tài: “Xây dựng hệ thống giám sát và điều khiển đèn giao thông thông minh”.

Mục tiêu của đề tài là xây dựng một hệ thống hoàn chỉnh ứng dụng Deep Learning (YOLO) để nhận diện phương tiện, kết hợp với kiến trúc Microservices (NestJS) và giao thức Socket.IO nhằm giải quyết bài toán giám sát và điều phối giao thông.

Chúng em xin gửi lời cảm ơn chân thành đến Thầy Đỗ Bá Lâm đã tận tình giảng dạy và trang bị những kiến thức nền tảng quý báu trong suốt quá trình học tập. Chính những kiến thức này là cơ sở quan trọng để nhóm có thể nghiên cứu, vận dụng và hoàn thành sản phẩm này.

Tuy nhiên, do thời gian thực hiện có hạn và kiến thức thực tế còn hạn chế, bài báo cáo chắc chắn không tránh khỏi những thiếu sót. Nhóm chúng em rất mong nhận được những ý kiến đóng góp, nhận xét từ Thầy để sản phẩm cũng như kiến thức của chúng em được hoàn thiện hơn.

Chúng em xin chân thành cảm ơn!

DANH SÁCH THÀNH VIÊN VÀ PHÂN CÔNG CÔNG VIỆC
NHÓM 13 – ĐỀ TÀI: HỆ THỐNG STREAMING VÀ XỬ LÝ ẢNH REALTIME

Thành viên	Vai trò	Nhiệm vụ chi tiết
Trần Quang Huy	Trưởng nhóm, AI, DevOps	- Quản lý: Khởi tạo Repo, thiết lập Quản lý tiến độ, Review & Merge code, chịu trách nhiệm chính về kiến trúc hệ thống - AI: Fine-tune model. Viết script xử lý luồng RTSP, thuật toán đếm xe, thuật toán điều khiển đèn - DevOps: Viết Dockerfile, Docker Compose để đóng gói toàn bộ dự án - Backend (Hỗ trợ): Định nghĩa cấu trúc JSON giao tiếp giữa AI và Server
Nguyễn Nhật Huy	Fullstack (BE + FE Support)	- Database: Thiết kế ERD, cấu hình Prisma ORM, Migration dữ liệu - API Core: Xây dựng module Authentication (JWT), CRUD Camera, Intersection, Users - Logic nghiệp vụ: Viết thuật toán điều khiển đèn tín hiệu (traffic.control.service) dựa trên dữ liệu từ AI - Socket Server: Cấu hình Gateway để nhận dữ liệu từ AI và broadcast cho Frontend - Frontend: Xử lý hiển thị Video Player (HLS/MJPEG) độ trễ thấp.
Nguyễn Văn Quang Huy	Fullstack (FE + BE Support)	- Frontend (Khó): Xử lý kết nối Socket.IO phía Client để cập nhật dữ liệu realtime - Data Visualization: Vẽ các biểu đồ thống kê (Chart.js/Recharts) động theo thời gian thực - Backend (Hỗ trợ): Viết một số API phụ trợ cho việc thống kê dữ liệu (History/Reporting) nếu Backend chính quá tải
Nguyễn Quang Huy	Frontend Dev	- UI/UX: Thiết kế Layout, cắt HTML/CSS (TailwindCSS), đảm bảo giao diện Responsive - Chức năng: Làm các trang Đăng nhập/Đăng ký, Trang quản trị (Admin Dashboard), Quản lý danh sách Camera/Nút giao - Testing: Kiểm thử giao diện, bắt lỗi form (Validation)

MỤC LỤC

LỜI NÓI ĐẦU	2
DANH SÁCH THÀNH VIÊN VÀ PHÂN CÔNG CÔNG VIỆC	3
CHƯƠNG I. GIỚI THIỆU CHUNG	7
1. Đặt vấn đề	7
2. Mục tiêu đề tài	7
3. Phạm vi và giới hạn đề tài	8
4. Phương pháp tiếp cận và giải pháp tổng quát	8
5. Đóng góp chính của đề tài	9
6. Cấu trúc báo cáo	9
CHƯƠNG 2. CƠ SỞ LÝ THUYẾT VÀ CÔNG NGHỆ	10
1. Công nghệ Xử lý ảnh và Trí tuệ nhân tạo (YOLO & OpenCV)	10
1.1. Thị giác máy tính (Computer Vision)	10
1.2. YOLO	10
1.3. OpenCV	10
1.4. ROI và đếm lưu lượng (Counting / Flow) từ CV	11
2. Công nghệ Backend (NestJS)	11
2.1. Giới thiệu NestJS	11
2.2. Đặc trưng kiến trúc của NestJS	12
3. Hệ quản trị Cơ sở dữ liệu (PostgreSQL & Prisma ORM)	12
3.1. PostgreSQL	12
3.2. Prisma ORM	12
4. Giao thức truyền thông thời gian thực (WebSocket & Socket.IO)	13
4.1. Web Socket	13
4.2. Socket.IO	13
5. Công nghệ Frontend	14
5.1. Framework chính: React 18	14
5.2. Điều hướng và phân quyền: React Router + Protected Routes	14
5.3. Quản lý trạng thái: Context-based State Management	14
5.4. Giao tiếp dữ liệu: REST API + Socket.IO Client	14
5.5. Hiển thị video giám sát: HLS + Hls.js	15
5.6. Trực quan hóa và xuất báo cáo	15

5.7. UI/UX tối ưu trải nghiệm	15
CHƯƠNG 3. PHÂN TÍCH YÊU CẦU	16
1. Yêu cầu chức năng	16
1.1. Phân hệ AI (Camera Node)	16
1.2. Phân hệ Backend.....	16
1.3. Phân hệ Frontend	16
2. Yêu cầu phi chức năng	17
3. Các ca sử dụng tiêu biểu.....	17
CHƯƠNG 4. THIẾT KẾ HỆ THỐNG VÀ TRIỂN KHAI	18
1. Tổng quan kiến trúc hệ thống	18
2. Thiết kế phân hệ AI	19
3. Thiết kế giao tiếp realtime và luồng dữ liệu điều khiển.....	20
4. Thiết kế cơ sở dữ liệu.....	20
CHƯƠNG 5. TRIỂN KHAI LÊN CÁC NỀN TẢNG	21
1. Tổng quan kiến trúc triển khai.....	21
2. Hạ tầng Frontend (Giao diện người dùng).....	21
3. Hạ tầng Backend (Dịch vụ ứng dụng)	21
4. Hạ tầng dữ liệu (Data Persistence Layer)	21
4.1. Cơ sở dữ liệu quan hệ (RDBMS).....	21
4.2. Bộ nhớ đệm và Quản lý phiên (Caching & Session)	22
5. Phân hệ Xử lý AI tại biên (Edge Computing).....	22
5.1. Mô hình xử lý	22
5.2. Giải pháp kết nối bảo mật (Secure Tunneling)	22
CHƯƠNG 6. THỬ NGHIỆM, ĐÁNH GIÁ VÀ KẾT LUẬN	23
1. Mục tiêu thử nghiệm	23
2. Môi trường và kịch bản thử nghiệm.....	23
2.1. Môi trường thử nghiệm	23
2.2. Kịch bản thử nghiệm	23
3. Kết quả và nhận xét	24
3.1. Kết quả giám sát realtime.....	24
3.2. Kết quả thống kê lịch sử theo phút.....	24
3.3. Kết quả điều khiển đèn.....	24
3.4. Kiểm tra ổn định khi mất kết nối	24

3.5. Hạn chế	24
GITHUB VÀ ĐÁNH GIÁ ĐÓNG GÓP CỦA TỪNG THÀNH VIÊN.....	25
QUẢN LÝ TIẾN ĐỘ CÔNG VIỆC	26
TÀI LIỆU THAM KHẢO.....	27

CHƯƠNG I. GIỚI THIỆU CHUNG

1. Đặt vấn đề

Trong bối cảnh đô thị hóa nhanh chóng tại Việt Nam, ùn tắc giao thông đang trở thành một vấn đề cấp thiết, gây thiệt hại lớn về kinh tế và lãng phí thời gian của xã hội. Một trong những nguyên nhân chính dẫn đến tình trạng này là hệ thống điều khiển đèn tín hiệu giao thông hiện tại phần lớn vẫn hoạt động theo chu kỳ thời gian cố định (Fixed-time control). Các hệ thống này không có khả năng linh hoạt điều chỉnh theo lưu lượng phương tiện thực tế, dẫn đến tình trạng: một làn đường vắng xe vẫn phải chờ đèn đỏ lâu, trong khi làn đường đông xe lại không đủ thời gian xanh để giải tỏa, gây ùn ứ và hiệu ứng “dồn nút” ở các khung giờ cao điểm.

Bên cạnh đó, vài năm gần đây, nhiều địa phương tại Việt Nam đang tăng cường triển khai các hệ thống camera giám sát giao thông, hướng tới mô hình đô thị thông minh. Việc “phủ” camera rộng hơn tạo ra một nguồn dữ liệu hình ảnh/video liên tục, mở ra cơ hội ứng dụng các kỹ thuật thị giác máy tính và trí tuệ nhân tạo để đo lường mật độ/lưu lượng theo thời gian thực. Nếu có thể khai thác tốt nguồn dữ liệu này, hệ thống đèn tín hiệu hoàn toàn có thể chuyển từ điều khiển cố định sang điều khiển thích nghi (adaptive), giúp tối ưu thời gian chờ, tăng hiệu quả giải tỏa và cải thiện trải nghiệm tham gia giao thông.

Từ thực tế đó, nhóm lựa chọn đề tài **Smart Traffic Lights** với mục tiêu xây dựng một hệ thống tích hợp gồm: (1) AI phân tích video từ camera để nhận diện và thống kê phương tiện, (2) Server trung tâm lưu trữ dữ liệu và tính toán logic điều khiển, (3) giao diện giám sát thời gian thực phục vụ vận hành và đánh giá.

2. Mục tiêu đề tài

Mục tiêu của đề tài là thiết kế và triển khai một hệ thống giám sát – điều khiển đèn tín hiệu dựa trên dữ liệu giao thông thời gian thực từ camera, cụ thể:

Mục tiêu kỹ thuật

- Nhận diện phương tiện từ luồng video camera (RTSP/file) bằng mô hình AI (YOLO).
- Tính toán các chỉ số giao thông theo thời gian thực: số xe trong vùng quan tâm, lưu lượng qua vạch, và thống kê theo phút (avg/max/flow).
- Truyền dữ liệu realtime về server, lưu trữ lịch sử để truy vấn phân tích.
- Hiện thị dashboard trực quan: video stream độ trễ thấp, biểu đồ lưu lượng, cảnh báo.

Mục tiêu nghiệp vụ

- Hỗ trợ người vận hành theo dõi tình trạng giao thông tại nút giao.
- Cung cấp dữ liệu đầu vào cho logic điều khiển thời gian xanh/đỏ theo mật độ (ở mức mô hình/giả lập trong phạm vi đồ án).

3. Phạm vi và giới hạn đề tài

Để phù hợp thời lượng và quy mô báo cáo, đề tài tập trung vào phạm vi sau:

- Phạm vi thực hiện
 - Mô phỏng/triển khai trên luồng camera RTSP hoặc video mẫu.
 - Nhận diện các nhóm phương tiện phổ biến (xe máy, ô tô, xe tải, xe buýt).
 - Thống kê realtime và theo phút; hiển thị dashboard và truy vấn lịch sử.
 - Logic điều khiển đèn ở mức thuật toán/giả lập (không yêu cầu tích hợp phần cứng đèn thực tế).
- Giới hạn
 - Chưa đi sâu theo hướng điều khiển tối ưu nâng cao (ví dụ RL/V2X) do giới hạn dữ liệu và thời gian.
 - Chất lượng nhận diện phụ thuộc điều kiện camera (góc đặt, ánh sáng, che khuất).
 - Đếm lưu lượng dựa trên ROI/vạch đếm và tracking đơn giản, chưa phải tracking đa mục tiêu “chuẩn công nghiệp”.

4. Phương pháp tiếp cận và giải pháp tổng quát

Nhóm tiếp cận theo hướng “end-to-end pipeline”, chia hệ thống thành ba phân hệ liên kết chặt chẽ:

- **Phân hệ AI (Camera Node):** OpenCV đọc luồng video → YOLO suy luận phát hiện xe → lọc ROI/đếm qua vạch → tổng hợp chỉ số → gửi dữ liệu realtime về server qua Socket.IO.
- **Phân hệ Backend:** nhận dữ liệu ingest → lưu CSDL (PostgreSQL/Prisma) → cung cấp API và kênh realtime cho dashboard → tính toán logic điều khiển theo chỉ số giao thông.
- **Phân hệ Frontend:** hiển thị video stream (HLS) → cập nhật realtime số liệu/biểu đồ → cảnh báo và xem lịch sử.

5. Đóng góp chính của đề tài

Trong phạm vi đề án, nhóm tập trung vào các đóng góp sau:

- Xây dựng được hệ thống **tích hợp hoàn chỉnh** từ camera → AI → server → dashboard theo thời gian thực.
- Thiết kế pipeline thị giác máy tính có **ROI + vạch đếm**, kèm thống kê theo phút phục vụ phân tích lịch sử.
- Tổ chức giao tiếp realtime theo hướng có thể mở rộng (phân kênh/namespace), phù hợp cho bài toán streaming nhiều camera.
- Cung cấp dashboard trực quan: xem video độ trễ thấp, theo dõi lưu lượng, xem lịch sử và cảnh báo.

6. Cấu trúc báo cáo

Báo cáo được tổ chức như sau:

- **Chương 1:** Giới thiệu chung (bối cảnh, mục tiêu, phạm vi).
- **Chương 2:** Cơ sở lý thuyết và công nghệ (AI thị giác, backend, CSDL, realtime, frontend).
- **Chương 3:** Phân tích yêu cầu hệ thống (yêu cầu chức năng và phi chức năng).
- **Chương 4:** Thiết kế và triển khai (kiến trúc, luồng dữ liệu, thiết kế các phân hệ).
- **Chương 5:** Triển khai hệ thống
- **Chương 6:** Thử nghiệm, đánh giá và kết luận (kịch bản demo, kết quả, hướng phát triển).

CHƯƠNG 2. CƠ SỞ LÝ THUYẾT VÀ CÔNG NGHỆ

1. Công nghệ Xử lý ảnh và Trí tuệ nhân tạo (YOLO & OpenCV)

1.1. Thị giác máy tính (Computer Vision)

Thị giác máy tính (Computer Vision) là lĩnh vực giúp máy tính “nhìn” và “hiểu” hình ảnh/video bằng cách biến dữ liệu pixel thành thông tin có ý nghĩa. Trong bài toán giao thông, Computer Vision thường được dùng cho các tác vụ như: phát hiện phương tiện (object detection), lọc theo vùng quan tâm (ROI), đếm xe/đo lưu lượng, và trích xuất chỉ số theo thời gian thực để phục vụ điều khiển đèn tín hiệu.

Trong hệ thống của nhóm, đầu vào là luồng video/RTSP từ camera (hoặc video mô phỏng), đầu ra là số lượng xe theo thời gian thực và các thống kê theo phút (average/max/samples/flow), sau đó được gửi về Backend qua Socket.IO để hiển thị và lưu trữ lịch sử.

1.2. YOLO

YOLO (You Only Look Once) là họ mô hình phát hiện vật thể theo hướng one-stage detector: mô hình dự đoán trực tiếp tọa độ bounding box và độ tin cậy chỉ trong một lần suy luận (single forward pass). Điều này phù hợp với bài toán giao thông cần xử lý realtime, vì hệ thống phải suy luận liên tục trên video để cập nhật trạng thái theo từng giây.

1.3. OpenCV

(1) Đọc luồng RTSP

- Sử dụng `cv2.VideoCapture(rtsp_or_path)` để mở camera.
- Thiết lập `CAP_PROP_BUFFERSIZE` để giảm độ trễ do buffer (tránh “xem lại quá khứ” khi realtime).

(2) Tối ưu tải tính toán

- Resize frame về kích thước nhỏ hơn trước khi infer (ví dụ 640×360 hoặc 512×288) để giảm CPU/GPU load.
- Bỏ qua frame (`frame_skip`): không infer mọi frame mà infer theo nhịp, vẫn đủ cho thống kê theo giây/phút.

(3) Hiển thị/Debug trực quan

- Vẽ bbox và score bằng `cv2.rectangle`, `cv2.putText`.
- Vẽ ROI polygon (`cv2.polyline`) và vẽ count-line (`cv2.line`) để debug đếm lưu lượng.

(4) Hỗ trợ cấu hình ROI bằng thao tác click

- Tool hỗ trợ click lên frame để lấy tọa độ điểm, in ra danh sách điểm để thiết lập vào cấu hình (ROI và 2 điểm cuối làm count-line).

1.4. ROI và đếm lưu lượng (Counting / Flow) từ CV

Ngoài “đếm số bbox”, nhóm triển khai các khái niệm thường dùng trong giao thông nhằm tạo chỉ số ổn định hơn:

(1) ROI (Region of Interest) – vùng quan tâm

- Chỉ các xe có tâm bbox (`cx,cy`) nằm trong ROI mới được tính.
- Kiểm tra “điểm trong đa giác” bằng `cv2.pointPolygonTest(...)`.

(2) Count-line – đếm xe theo hướng qua vạch

- Mỗi xe được gán một ID theo dạng tracking đơn giản dựa trên centroid.
- Khi centroid di chuyển từ frame trước sang frame sau và cắt qua đường đếm, hệ thống tăng `flow_count`.

(3) Tổng hợp số liệu theo phút

- Lưu các mẫu theo giây (`samples`), tính `vehicles_avg`, `vehicles_max`, và (nếu bật) `flow_count` rồi gửi về backend mỗi khi sang phút mới.

2. Công nghệ Backend (NestJS)

2.1. Giới thiệu NestJS

Hệ thống Backend của đề tài được xây dựng dựa trên NestJS – một framework Node.js tiên bộ, định hướng phát triển ứng dụng theo chuẩn hiện đại và ưu tiên TypeScript nhằm tăng tính an toàn kiểu dữ liệu (type-safe) và giảm lỗi khi mở rộng hệ thống. Việc sử dụng TypeScript cũng giúp mã nguồn rõ ràng hơn, thuận lợi cho phát triển theo nhóm và nâng cao khả năng bảo trì trong các dự án có quy mô vừa–lớn.

2.2. Đặc trưng kiến trúc của NestJS

NestJS khuyến khích kiến trúc tách lớp (separation of concerns) và tổ chức theo cấu trúc rõ ràng, trong đó các thành phần chính thường gồm:

- **Module:** đóng vai trò “khối chức năng” (feature unit), gom các thành phần liên quan vào cùng một phạm vi, giúp hệ thống dễ mở rộng theo từng chức năng.
- **Controller:** chịu trách nhiệm giao tiếp (expose API), nhận request và trả response.
- **Service/Provider:** chứa logic nghiệp vụ cốt lõi, dễ tái sử dụng và dễ test.

Bên cạnh đó, NestJS hỗ trợ Dependency Injection (DI) giúp giảm phụ thuộc cứng giữa các thành phần, từ đó tăng tính linh hoạt và dễ kiểm thử khi hệ thống phát triển lớn dần.

3. Hệ quản trị Cơ sở dữ liệu (PostgreSQL & Prisma ORM)

3.1. PostgreSQL

PostgreSQL là một hệ quản trị cơ sở dữ liệu quan hệ (RDBMS) mạnh mẽ, mã nguồn mở, được sử dụng rộng rãi trong các hệ thống backend cần độ tin cậy cao. CSDL quan hệ phù hợp cho bài toán giao thông vì dữ liệu có cấu trúc rõ ràng và liên kết chặt chẽ (ví dụ: nút giao – camera – thống kê theo phút – log trạng thái đèn).

Trong hệ thống của đề tài, PostgreSQL đóng vai trò lưu trữ dữ liệu lịch sử và dữ liệu nghiệp vụ phục vụ giám sát/điều khiển, đặc biệt là dữ liệu theo thời gian như thống kê theo phút và log vận hành. Nhờ đó, hệ thống có thể:

- Truy vấn lịch sử để hiển thị dashboard và phân tích xu hướng.
- Đảm bảo tính nhất quán dữ liệu giữa các thực thể (quan hệ/khóa ngoại).
- Mở rộng khi số lượng camera/nút giao tăng lên.

3.2. Prisma ORM

Để tương tác với PostgreSQL, nhóm sử dụng Prisma – một ORM (Object–Relational Mapping) hiện đại. Prisma cung cấp cơ chế làm việc với CSDL theo hướng “schema-first”, mô tả mô hình dữ liệu trong file schema, từ đó tự động tạo Prisma Client giúp thao tác CSDL theo kiểu type-safe (an toàn kiểu dữ liệu).

ORM giúp gì so với SQL thuần?

Thay vì viết nhiều câu lệnh SQL thủ công, ORM cho phép lập trình viên thao tác theo mô hình “đối tượng” và các hàm truy vấn có cấu trúc:

- **Dễ đọc & dễ bảo trì:** truy vấn rõ ràng, giảm “SQL rải rác”.
- **Giảm lỗi do sai kiểu dữ liệu:** nhờ type-safe, nhiều lỗi được phát hiện sớm trong quá trình phát triển.
- **Hỗ trợ migration thuận tiện:** thay đổi cấu trúc bảng theo phiên bản, dễ triển khai và đồng bộ môi trường.

4. Giao thức truyền thông thời gian thực (WebSocket & Socket.IO)

4.1. Web Socket

WebSocket là một giao thức truyền thông cho phép thiết lập kết nối duy trì (persistent connection) giữa Client và Server, hỗ trợ giao tiếp hai chiều (full-duplex). Khác với HTTP truyền thống (request/response rời rạc), WebSocket giúp Server có thể đẩy dữ liệu ngay lập tức xuống Client mà không cần Client liên tục “polling”, từ đó giảm độ trễ và rất phù hợp với các bài toán giám sát/điều khiển thời gian thực.

Trong đề tài, Backend sử dụng WebSocket để đảm bảo độ trễ thấp cho các tác vụ giám sát lưu lượng và cập nhật trạng thái điều khiển.

4.2. Socket.IO

Mặc dù WebSocket là giao thức nền tảng, nhóm triển khai theo hướng thực dụng bằng Socket.IO – một thư viện giúp xây dựng hệ thống realtime ổn định hơn trong môi trường mạng không đảm bảo (mất kết nối, reconnect, timeout...), đồng thời cung cấp các cơ chế tổ chức luồng dữ liệu quan trọng cho bài toán streaming như namespace và room.

(1) Ổn định kết nối và xử lý mạng không ổn định

Socket.IO hỗ trợ cơ chế tự reconnect, giúp các node AI và dashboard duy trì kết nối ổn định khi mạng chập chờn hoặc backend restart.

(2) Namespace – tách kênh realtime theo mục đích

Chia namespace giúp tách luồng ingest (AI gửi dữ liệu liên tục) và luồng dashboard (người dùng theo dõi), tối ưu hiệu năng và kiểm soát truy cập tốt hơn.

(3) Room – gom nhóm client để broadcast đúng đối tượng

Room giúp nhóm các client theo ngữ cảnh (theo camera/intersection), từ đó backend có thể broadcast đúng nơi cần nhận, giảm băng thông và tăng khả năng mở rộng.

5. Công nghệ Frontend

Frontend (Traffic Monitor) được xây dựng theo mô hình Single Page Application (SPA), cho phép cập nhật giao diện liên tục mà không cần tải lại trang, phù hợp với yêu cầu hiển thị dữ liệu giao thông theo thời gian thực. Kiến trúc frontend kết hợp quản lý trạng thái toàn cục bằng Context và tối ưu tải trang bằng cơ chế tách module/lazy loading.

5.1. Framework chính: React 18

Frontend sử dụng React 18 theo hướng functional components và hooks (useState, useEffect, useContext...), giúp tổ chức UI rõ ràng và cập nhật dữ liệu nhanh. Cơ chế Virtual DOM giúp UI phản hồi tốt với dữ liệu realtime (lưu lượng xe, trạng thái đèn, cảnh báo).

5.2. Điều hướng và phân quyền: React Router + Protected Routes

Hệ thống sử dụng React Router để điều hướng trong SPA. Các trang nghiệp vụ (dashboard, history, quản trị camera/nút giao) được bảo vệ bởi cơ chế "Protected Routes", đảm bảo chỉ người dùng đã đăng nhập mới truy cập được.

5.3. Quản lý trạng thái: Context-based State Management

Frontend tổ chức state toàn cục theo các context chính:

- **Auth Context:** quản lý đăng nhập, token, thông tin người dùng và trạng thái phiên.
- **Traffic Context:** quản lý dữ liệu giao thông theo thời gian thực, danh sách camera/nút giao và các trạng thái hiển thị trên dashboard.

Cách làm này giúp tránh truyền props phức tạp và giữ dữ liệu nhất quán giữa nhiều component.

5.4. Giao tiếp dữ liệu: REST API + Socket.IO Client

Frontend vừa gọi REST API để lấy dữ liệu nền (danh sách camera/nút giao, lịch sử thống kê), vừa nhận dữ liệu realtime qua Socket.IO để cập nhật dashboard tức thời. Việc chia kênh theo namespace giúp tách luồng dữ liệu ingest và luồng dashboard, phù hợp cho streaming nhiều camera.

5.5. Hiện thị video giám sát: HLS + Hls.js

Do trình duyệt không phát trực tiếp RTSP, hệ thống sử dụng cơ chế chuyển RTSP sang HLS ở tầng streaming. Trên frontend, Hls.js được dùng để phát HLS ổn định, hỗ trợ chế độ độ trễ thấp và tương thích đa trình duyệt.

5.6. Trực quan hóa và xuất báo cáo

- **Biểu đồ/Phân tích:** sử dụng thư viện biểu đồ (ví dụ Recharts) để biểu diễn lưu lượng theo thời gian, so sánh nhiều camera/nút giao.
- **Xuất dữ liệu:** hỗ trợ xuất thống kê sang file Excel (XLSX) phục vụ báo cáo/đánh giá.

5.7. UI/UX tối ưu trải nghiệm

Giao diện được thiết kế theo hướng dễ quan sát cho người vận hành: bố cục dashboard trực quan, hỗ trợ dark/light mode, hiển thị cảnh báo tắc đường hoặc trạng thái offline của camera/AI. Đồng thời, việc tách module và tối ưu tải trang giúp trải nghiệm mượt hơn khi mở nhiều thành phần (video + biểu đồ + bảng số liệu) cùng lúc.

CHƯƠNG 3. PHÂN TÍCH YÊU CẦU

1. Yêu cầu chức năng

1.1. Phân hệ AI (Camera Node)

- Kết nối và đọc luồng video từ **Camera RTSP** hoặc **file video** để phục vụ thử nghiệm/mô phỏng.
- Nhận diện phương tiện giao thông theo các lớp mục tiêu: **xe máy, ô tô, xe tải, xe buýt**.
- Tính toán **logic điều khiển đèn tín hiệu** dựa trên dữ liệu lưu lượng (điều chỉnh thời gian xanh/đỏ theo mật độ).
- Hỗ trợ cấu hình **ROI (Region of Interest)** để chỉ tính các phương tiện thuộc khu vực/làn quan tâm.
- Đếm **lưu lượng xe đi qua vạch (count-line)** theo hướng quy định nhằm thu được chỉ số **flow_count**.
- Tính toán và gửi dữ liệu **realtime** theo chu kỳ **~1 giây** (số xe hiện tại, trạng thái tổng quát).
- Tổng hợp thống kê theo phút (ví dụ: **vehicles_avg, vehicles_max, samples, flow_count**) và gửi về server để lưu trữ lịch sử.
- Tự động **reconnect** khi backend/mạng bị ngắt để đảm bảo pipeline hoạt động liên tục.

1.2. Phân hệ Backend

- Hỗ trợ **xác thực người dùng**: đăng ký, đăng nhập, quên mật khẩu.
- Quản lý danh sách **Nút giao (Intersection)** và **Camera**, bao gồm thông tin và cấu hình liên quan.
- Tiếp nhận dữ liệu **realtime** từ **AI** qua Socket.IO (kênh ingest) và cập nhật trạng thái mới nhất.
- Lưu trữ dữ liệu thống kê theo phút và các log vận hành vào CSDL để phục vụ truy vấn lịch sử/đánh giá.
- Cung cấp **REST API** cho frontend (danh sách camera/nút giao, dữ liệu lịch sử, tài khoản...).
- Cung cấp kênh **realtime** cho frontend để cập nhật dashboard tức thời.
- Giám sát trạng thái kết nối AI node; khi AI offline phải cập nhật trạng thái và cho phép frontend hiển thị cảnh báo.

1.3. Phân hệ Frontend

- Xem **video stream trực tiếp** từ camera với độ trễ thấp.
- Xem **dashboard thống kê lưu lượng** theo thời gian thực (số xe, mật độ, trạng thái theo camera/nút giao).
- Xem **lịch sử thống kê** (theo khoảng thời gian) và biểu đồ hóa để phân tích.

- Hiện thị **cảnh báo tắc đường/bất thường** (vượt ngưỡng lưu lượng, camera/AI offline...).
- Hỗ trợ giao diện quản trị cơ bản (tùy phân quyền): cấu hình camera/nút giao, xem danh sách thiết bị.

2. Yêu cầu phi chức năng

Tính thời gian thực (Real-time): Độ trễ từ lúc xe đi qua camera đến khi hiển thị trên dashboard < 2 giây.

- **Hiệu năng AI:** AI node phải xử lý tối thiểu **15 FPS** (hoặc tốc độ tương đương đủ để đếm ổn định trong video realtime).
- **Tính ổn định (Reliability):** Backend phải hoạt động ổn định khi AI node mất kết nối; hệ thống phải tự phục hồi kết nối (reconnect) và không gây sập dịch vụ.
- **Tính nhất quán dữ liệu:** Dữ liệu thống kê theo phút và log điều khiển phải được lưu bền vững, hạn chế mất dữ liệu khi restart.
- **Bảo mật:** Các chức năng nghiệp vụ trên dashboard phải yêu cầu xác thực; tách kênh ingest và kênh user để kiểm soát truy cập.
- **Khả năng mở rộng:** Có thể mở rộng theo số lượng camera/nút giao (scale theo chiều ngang) mà không thay đổi kiến trúc lớn.
- **Khả năng bảo trì:** Mã nguồn và cấu hình phải module hóa/cấu hình hóa để dễ chỉnh sửa khi mở rộng hoặc thay đổi yêu cầu.

3. Các ca sử dụng tiêu biểu

UC-01: Giám sát giao thông thời gian thực

- Người dùng đăng nhập dashboard → chọn nút giao/camera → xem video stream + số liệu realtime → hệ thống cập nhật liên tục qua Socket.IO.

UC-02: Xem lịch sử và phân tích lưu lượng

- Người dùng chọn khoảng thời gian → hệ thống truy vấn dữ liệu theo phút trong CSDL → hiển thị biểu đồ và bảng thống kê → hỗ trợ xuất dữ liệu (nếu có).

UC-03: Vận hành/điều khiển đèn (theo logic tự động)

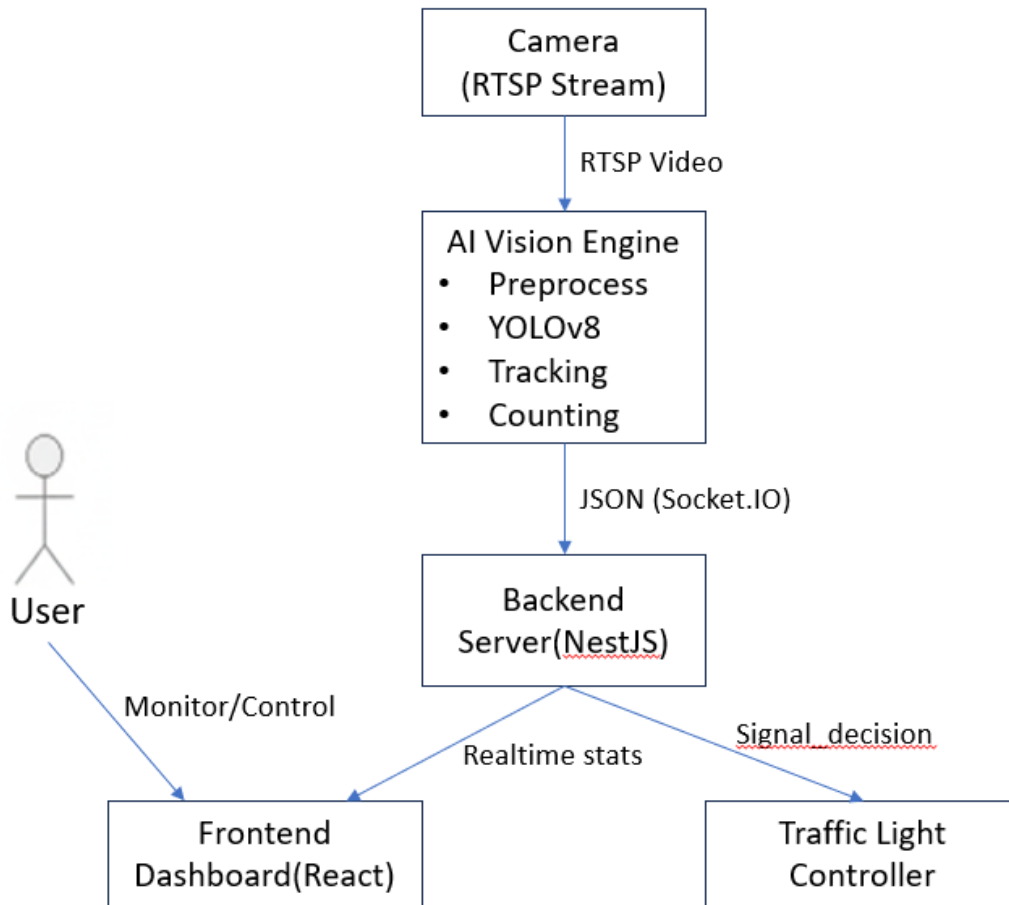
- Backend nhận dữ liệu lưu lượng → tính toán thời gian pha phù hợp → cập nhật trạng thái đèn và log vận hành → dashboard hiển thị trạng thái hiện tại.

UC-04: Cảnh báo bất thường

Khi lưu lượng vượt ngưỡng hoặc AI/camera offline → backend phát cảnh báo → dashboard hiển thị thông báo để người vận hành kịp thời xử lý.

CHƯƠNG 4. THIẾT KẾ HỆ THỐNG VÀ TRIỂN KHAI

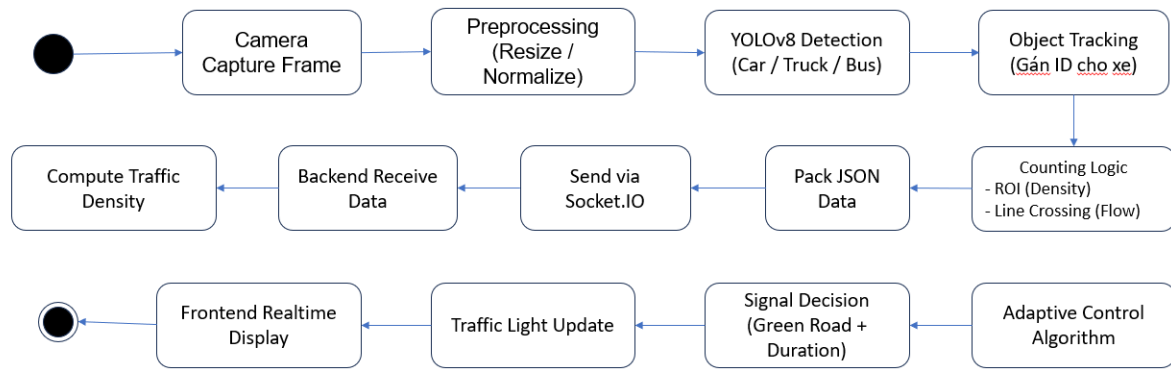
1. Tổng quan kiến trúc hệ thống



Hình 4.1. Kiến trúc tổng quan hệ thống Smart Traffic Lights

Hệ thống được thiết kế theo kiến trúc 3 lớp chính gồm **Camera/AI Node – Backend Server – Frontend Dashboard**, kết hợp với khối **Traffic Light Controller**. Camera cung cấp luồng RTSP, AI Vision Engine thực hiện tiền xử lý, nhận diện phương tiện bằng YOLOv8, theo dõi đối tượng và đếm lưu lượng. Dữ liệu sau đó được đóng gói dạng JSON và truyền về Backend qua Socket.IO. Backend chịu trách nhiệm lưu trữ dữ liệu, cung cấp realtime stats cho Dashboard, đồng thời thực hiện logic ra quyết định đèn (signal decision) để cập nhật trạng thái điều khiển. Người dùng giám sát/điều khiển thông qua giao diện Dashboard.

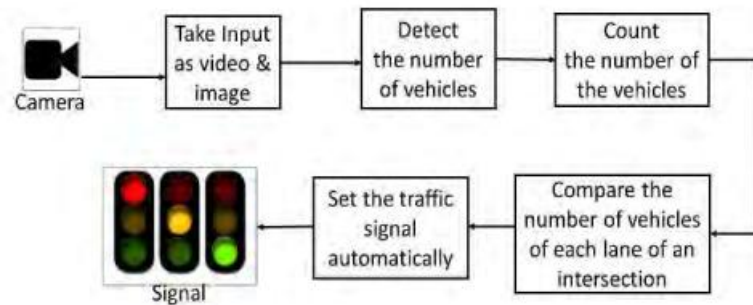
2. Thiết kế phân hệ AI



Hình 4.2. Pipeline xử lý AI tại Camera Node

Đầu tiên, hệ thống đọc frame từ RTSP/video file. Frame được tiền xử lý (resize/normalize) để phù hợp đầu vào mô hình. Tiếp theo, YOLOv8 thực hiện phát hiện phương tiện (xe máy/ô tô/xe tải/xe buýt). Kết quả detection được đưa qua bước tracking nhằm gán ID ổn định cho từng xe giữa các frame, phục vụ bài toán đếm.

Ở bước counting, hệ thống áp dụng hai cơ chế: **(i) ROI-based density** để lọc phương tiện nằm trong vùng quan tâm và ước lượng mật độ; **(ii) line-crossing flow** để đếm lưu lượng xe đi qua vạch theo hướng xác định. Cuối cùng, dữ liệu được đóng gói JSON và gửi về Backend qua Socket.IO để lưu trữ và hiển thị realtime.



Hình 4.3. Quy trình điều khiển đèn tín hiệu

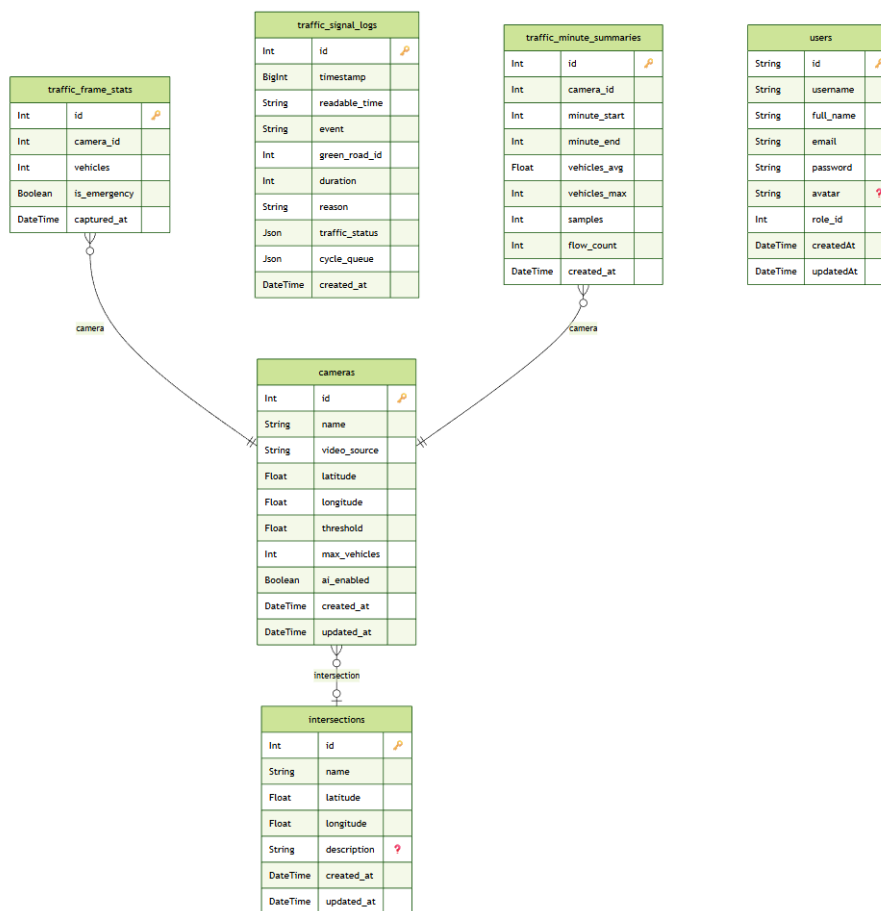
Hệ thống nhận đầu vào là video/hình ảnh từ camera, sau đó thực hiện phát hiện và đếm số lượng phương tiện trên từng làn/nhánh của nút giao. Các số liệu đếm được được dùng để so sánh mức độ ùn tắc tương đối giữa các hướng (ví dụ hướng nào có mật độ/lưu lượng cao hơn sẽ được ưu tiên). Dựa trên kết quả so sánh, bộ điều khiển sẽ tự động quyết định trạng thái đèn (xanh/đỏ) và điều chỉnh thời gian pha nhằm phân bổ thời gian xanh hợp lý, giảm thời gian chờ ở hướng đông xe và tăng hiệu quả giải tỏa giao thông.

3. Thiết kế giao tiếp realtime và luồng dữ liệu điều khiển

Luồng dữ liệu realtime được tổ chức theo hướng tách bạch: AI Node gửi dữ liệu lên Backend theo kênh ingest (ưu tiên tốc độ và ổn định), sau đó Backend phát dữ liệu cho Dashboard theo kênh dành cho người dùng (có kiểm soát truy cập). Dữ liệu truyền gồm hai nhóm chính: **realtime theo giây** (phục vụ hiển thị tức thời và ra quyết định nhanh) và **tổng hợp theo phút** (phục vụ phân tích lịch sử và đánh giá hiệu quả điều khiển).

Trên cơ sở thống kê lưu lượng/mật độ, AI thực hiện **signal decision** (chọn hướng được ưu tiên xanh và thời lượng tương ứng) và phát lệnh cập nhật trạng thái đèn. Trạng thái mới được đồng bộ trở lại Dashboard để người dùng quan sát và giám sát vận hành.

4. Thiết kế cơ sở dữ liệu



Hình 4.3. Lược đồ cơ sở dữ liệu hệ thống

CHƯƠNG 5. TRIỂN KHAI LÊN CÁC NỀN TẢNG

1. Tổng quan kiến trúc triển khai

Để đảm bảo tính sẵn sàng cao (High Availability), khả năng mở rộng (Scalability) và tối ưu hóa chi phí vận hành, nhóm thực hiện đã đề xuất mô hình triển khai **Hybrid Cloud**. Hệ thống kết hợp giữa các dịch vụ điện toán đám mây (Cloud Computing) cho các tác vụ quản lý dữ liệu và mô hình điện toán biên (Edge Computing) cho các tác vụ xử lý trí tuệ nhân tạo (AI) yêu cầu độ trễ thấp.

Sơ đồ triển khai được phân chia thành các phân hệ hạ tầng như sau:

2. Hạ tầng Frontend (Giao diện người dùng)

Phân hệ Frontend (ReactJS) được triển khai trên nền tảng **Vercel**. Đây là giải pháp tối ưu cho các ứng dụng web hiện đại (Single Page Application - SPA) với các đặc điểm kỹ thuật sau:

- **Cơ chế phân phối nội dung (Global CDN):** Mã nguồn tĩnh (Static Assets) được phân phối qua mạng lưới máy chủ toàn cầu (Edge Network), giúp giảm thiểu độ trễ tải trang cho người dùng cuối bất kể vị trí địa lý.
- **Quy trình CI/CD tự động:** Hệ thống tích hợp quy trình tích hợp và triển khai liên tục (Continuous Deployment). Mỗi thay đổi trong mã nguồn (Git Commit) đều kích hoạt quy trình tự động build và deploy phiên bản mới, đảm bảo tính nhất quán của sản phẩm.

3. Hạ tầng Backend (Dịch vụ ứng dụng)

Phân hệ Backend (NestJS) được đóng gói theo chuẩn **Containerization** (sử dụng Docker) và triển khai trên nền tảng **Render Cloud**.

- **Môi trường thực thi (Runtime Environment):** Việc sử dụng Docker Container đảm bảo sự đồng nhất tuyệt đối giữa môi trường phát triển (Development) và môi trường vận hành (Production), loại bỏ các lỗi phát sinh do sai lệch cấu hình hệ thống.
- **Quản lý tài nguyên:** Render cung cấp khả năng tự động khởi động lại dịch vụ khi có sự cố (Auto-healing) và giám sát tài nguyên (CPU/RAM) theo thời gian thực.

4. Hạ tầng dữ liệu (Data Persistence Layer)

Tầng dữ liệu được thiết kế tách biệt để đảm bảo hiệu năng và an toàn dữ liệu:

4.1. Cơ sở dữ liệu quan hệ (RDBMS)

Hệ thống sử dụng **Supabase** - một dịch vụ quản lý cơ sở dữ liệu dựa trên **PostgreSQL**.

- **Vai trò:** Lưu trữ dữ liệu nghiệp vụ cốt lõi, thông tin người dùng và lịch sử hoạt động.
- **Lý do lựa chọn:** Supabase cung cấp khả năng mở rộng linh hoạt, cơ chế sao lưu tự động (Point-in-time Recovery) và bảo mật dữ liệu ở mức Row Level Security (RLS).

4.2. Bộ nhớ đệm và Quản lý phiên (Caching & Session)

Hệ thống tích hợp **Upstash** sử dụng công nghệ **Redis** theo kiến trúc **Serverless**.

- **Vai trò:** Lưu trữ Session đăng nhập, Cache các truy vấn API tần suất cao và làm hàng đợi (Message Broker) cho các tác vụ nền.
- **Ưu điểm:** Kiến trúc Serverless giúp hệ thống không cần duy trì máy chủ Redis liên tục, tối ưu chi phí dựa trên lưu lượng truy cập thực tế (Pay-per-request) đồng thời vẫn đảm bảo độ trễ truy xuất dữ liệu ở mức mili-giây.

5. Phân hệ Xử lý AI tại biên (Edge Computing)

Khác với các kiến trúc Cloud-centric truyền thống, module xử lý hình ảnh và AI được triển khai theo mô hình **Edge Computing** (Điện toán biên) kết hợp với giải pháp **Cloudflare Tunnel**.

5.1. Mô hình xử lý

Toàn bộ quá trình thu nhận tín hiệu từ Camera và xử lý mô hình AI (Object Detection) được thực hiện tại thiết bị cục bộ (Local Server/Edge Device).

- **Giảm tải băng thông:** Chỉ gửi dữ liệu kết quả (Metadata/JSON) lên Server thay vì truyền tải toàn bộ luồng video (Video Streaming) liên tục, giúp tiết kiệm băng thông mạng và giảm tải cho Backend Server.
- **Tối ưu độ trễ:** Việc xử lý tại biên giúp hệ thống phản ứng tức thời với các sự kiện giao thông mà không phụ thuộc vào tốc độ đường truyền Internet.

5.2. Giải pháp kết nối bảo mật (Secure Tunneling)

Để kết nối Edge Device với hệ thống Cloud Backend mà không làm lộ hạ tầng mạng nội bộ, nhóm sử dụng công nghệ **Cloudflare Tunnel (Zero Trust Network)**.

- **Cơ chế:** Thiết lập một đường hầm mã hóa (Encrypted Tunnel) từ thiết bị biên ra Internet thông qua hạ tầng của Cloudflare.
- **Bảo mật:** Giải pháp này loại bỏ hoàn toàn yêu cầu mở Port (Port Forwarding) trên Router, ngăn chặn các nguy cơ tấn công từ bên ngoài vào hệ thống Camera nội bộ, đảm bảo tính riêng tư và an toàn thông tin.

CHƯƠNG 6. THỬ NGHIỆM, ĐÁNH GIÁ VÀ KẾT LUẬN

1. Mục tiêu thử nghiệm

- Đánh giá khả năng **nhận diện/đếm phương tiện** từ luồng video camera (RTSP/file).
- Đánh giá khả năng **truyền dữ liệu thời gian thực** từ AI → Backend → Frontend.
- Đánh giá khả năng **lưu trữ và truy vấn dữ liệu lịch sử** (thống kê theo phút).
- Kiểm tra tính đúng đắn của **logic quyết định đèn** (signal decision) dựa trên mật độ/lưu lượng.

2. Môi trường và kịch bản thử nghiệm

2.1. Môi trường thử nghiệm

Hệ thống được triển khai theo kiến trúc gồm AI Service (Python), Backend (NestJS) và Frontend (React). Camera được mô phỏng bằng video hoặc RTSP stream nội bộ, dữ liệu được truyền qua Socket.IO và lưu vào PostgreSQL.

2.2. Kịch bản thử nghiệm

Nhóm thực hiện các kịch bản kiểm thử tiêu biểu sau:

Kịch bản 1: Giám sát realtime 1 camera

- Khởi động camera stream (RTSP/video).
- AI Service đọc luồng, nhận diện và đếm phương tiện theo từng giây.
- Backend nhận dữ liệu realtime và phát về dashboard.
- Frontend hiển thị số liệu cập nhật liên tục và video stream độ trễ thấp.

Kịch bản 2: Thống kê theo phút và truy vấn lịch sử

- AI Service tổng hợp vehicles_avg, vehicles_max, flow_count, samples theo phút.
- Backend lưu dữ liệu vào bảng thống kê theo phút.
- Frontend truy vấn và hiển thị biểu đồ theo khoảng thời gian (History/Analysis).

Kịch bản 3: Kiểm tra điều khiển đèn theo mật độ

- Tạo tình huống mô phỏng 2 hướng có lưu lượng khác nhau (đông/ít).
- Backend nhận thống kê và đưa ra quyết định: ưu tiên hướng đông xe, điều chỉnh thời gian pha xanh/đỏ.
- Trạng thái đèn được cập nhật realtime lên dashboard và ghi log.

Kịch bản 4: Kiểm tra tính ổn định khi mất kết nối

- Cố tình ngắt kết nối AI Service hoặc backend trong thời gian ngắn.
- Quan sát cơ chế reconnect và khả năng hệ thống phục hồi, đồng bộ lại dữ liệu.

3. Kết quả và nhận xét

3.1. Kết quả giám sát realtime

Trong quá trình chạy thử nghiệm, hệ thống hiển thị realtime số lượng xe theo từng giây trên dashboard. Việc dùng Socket.IO giúp dữ liệu cập nhật liên tục; video stream (HLS) cho phép người vận hành quan sát trực quan tình trạng tại camera.

Nhận xét: Luồng dữ liệu realtime hoạt động ổn định, tuy nhiên do nhóm không có điều kiện để test trên cam thật, cộng với việc giả lập cam nên đôi lúc kết nối còn chậm, nhưng tạm chấp nhận được với quy mô bài toán.

3.2. Kết quả thống kê lịch sử theo phút

Dữ liệu tổng hợp theo phút được lưu vào cơ sở dữ liệu và truy vấn lại trên trang lịch sử. Biểu đồ lưu lượng thể hiện rõ sự thay đổi theo thời gian và hỗ trợ so sánh giữa các camera/nút giao.

Nhận xét: Tổng hợp theo phút giúp giảm nhiều so với đếm theo frame, phù hợp để phân tích xu hướng và đánh giá điều khiển. Sau này nếu có thể scale lên theo mỗi giờ cũng đơn giản

3.3. Kết quả điều khiển đèn

Khi mô phỏng các hướng có lưu lượng khác nhau, backend đưa ra quyết định ưu tiên hướng có mật độ/lưu lượng lớn hơn bằng cách tăng thời gian xanh tương ứng. Quyết định được ghi log và đồng bộ lên dashboard.

Nhận xét: Logic điều khiển ở mức đồ án cho thấy khả năng “thích nghi” theo dữ liệu. Tuy nhiên, đây mới là mô hình điều khiển cơ bản; để tối ưu hơn cần thêm ràng buộc an toàn và chiến lược tối ưu nâng cao.

3.4. Kiểm tra ổn định khi mất kết nối

Khi ngắt kết nối tạm thời, AI Service có thể kết nối lại và tiếp tục gửi dữ liệu. Dashboard cập nhật trạng thái offline/online theo tình trạng kết nối.

Nhận xét: Cơ chế reconnect giúp hệ thống bền vững hơn trong môi trường mạng thực tế.

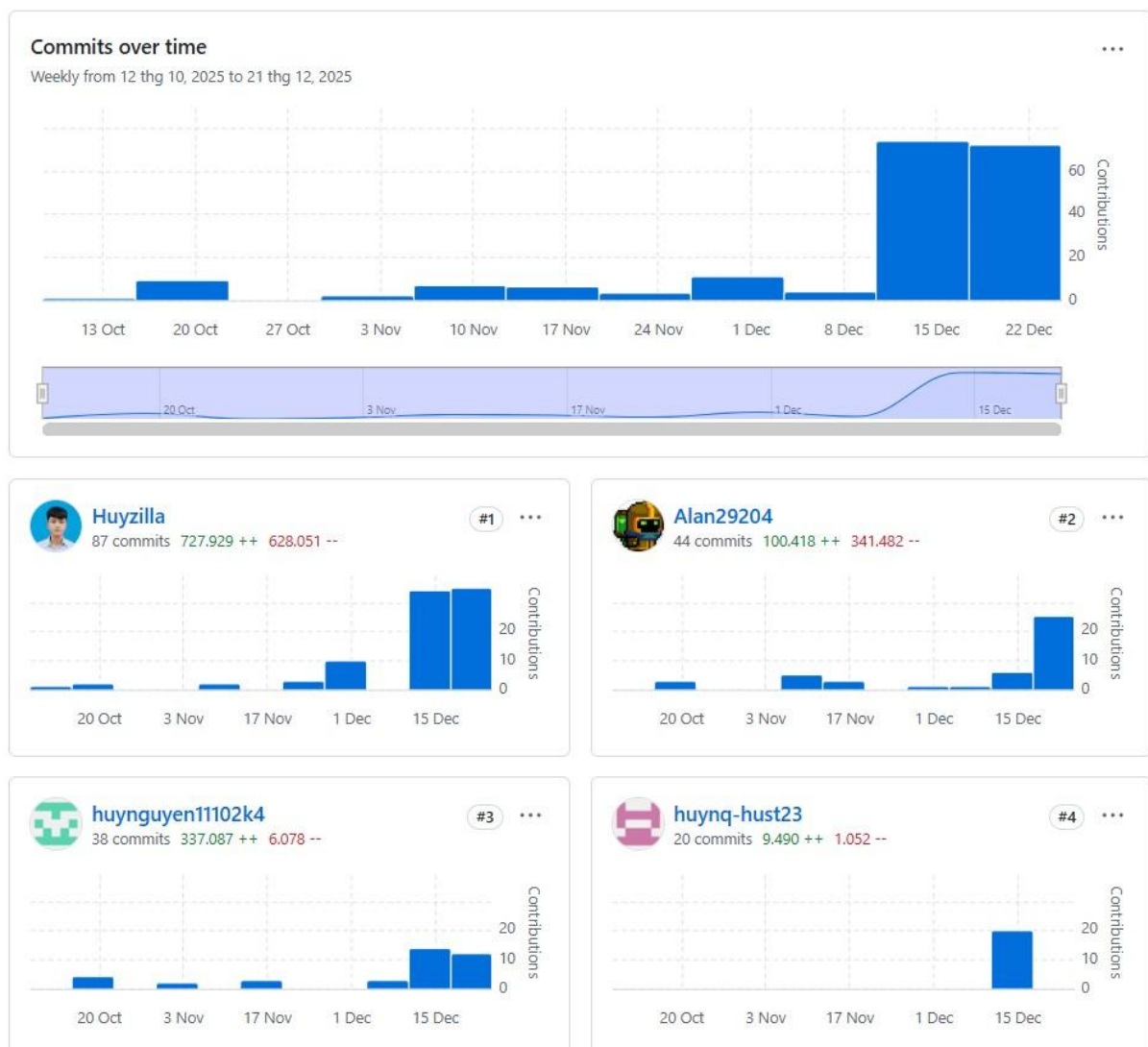
3.5. Hạn chế

- Chất lượng phát hiện/đếm phụ thuộc mạnh vào góc camera, ánh sáng và che khuất.

- Tracking/đếm dựa trên centroid và line-crossing có thể sai trong tình huống mật độ quá cao hoặc xe đi quá sát nhau.
- Logic điều khiển hiện ở mức thích nghi cơ bản theo lưu lượng; chưa tối ưu theo nhiều mục tiêu (giảm tổng delay, tối ưu chu kỳ toàn nút giao...).
- Trong quá trình thử nghiệm, nhóm giả lập camera bằng luồng RTSP (video loop), do đó có thể xuất hiện hiện tượng trễ (lag), jitter và mất gói, dẫn đến kết nối không ổn định. Điều này làm giảm FPS thực tế, tăng độ trễ end-to-end và có thể ảnh hưởng đến độ mượt của dữ liệu realtime hiển thị trên dashboard.

GITHUB VÀ ĐÁNH GIÁ ĐÓNG GÓP CỦA TỪNG THÀNH VIÊN

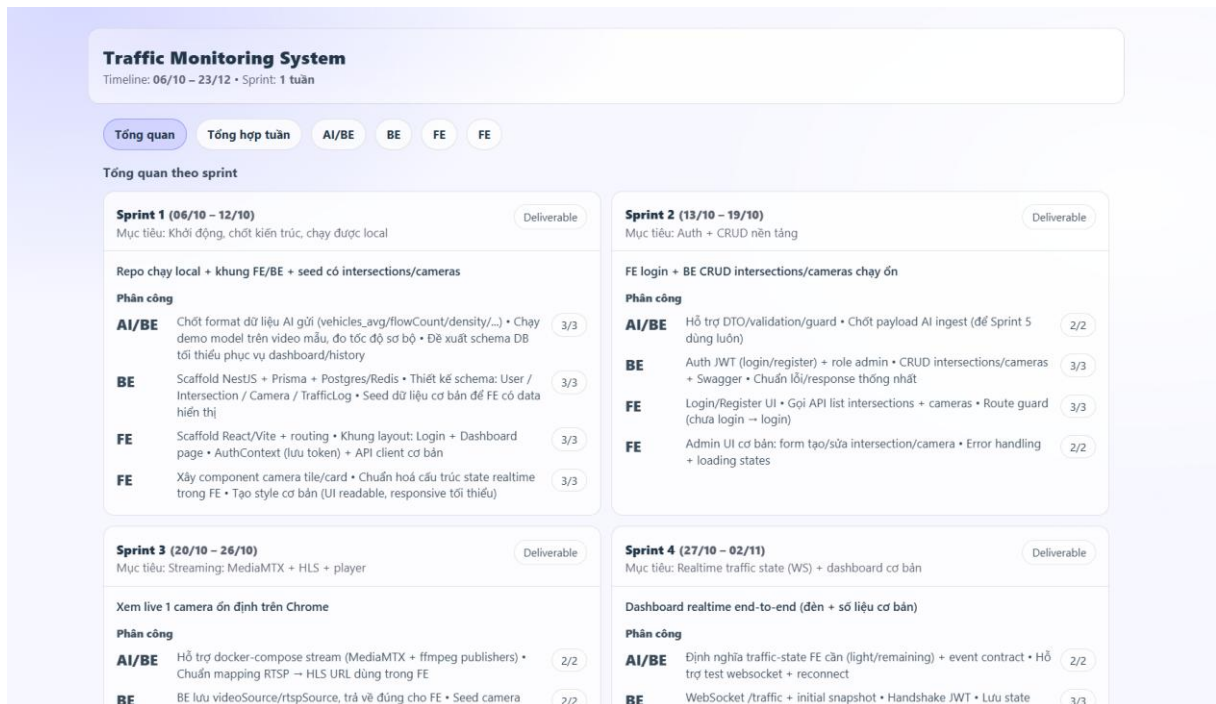
Github: https://github.com/Huyzilla/IT4409_QuadHuy



Hình 4.4. Đóng góp của từng thành viên

QUẢN LÝ TIẾN ĐỘ CÔNG VIỆC

Link: <https://process-management-seven.vercel.app/>



Hình 4.5: Tiến độ công việc được đánh giá theo từng tuần

TÀI LIỆU THAM KHẢO

- [2] Ultralytics, *Ultralytics YOLO Documentation (YOLOv8)*.
- [3] OpenCV Team, *OpenCV Documentation*.
- [4] ONNX Runtime, *ONNX Runtime Documentation*.
- [5] NestJS, *NestJS Documentation*.
- [6] Prisma, *Prisma ORM Documentation*.
- [7] PostgreSQL Global Development Group, *PostgreSQL Documentation*.
- [8] Socket.IO, *Socket.IO Documentation*.
- [9] React, *React Documentation*.
- [10] React Router, *React Router Documentation*.